

Artificial Pacemaker Simulation

Summary Report

Objective

To simulate the operation of an artificial pacemaker that monitors ECG signals and delivers pacing pulses when the heart rate falls below a safe limit.

1. Signal Generation

A synthetic ECG-like waveform is created:

```
ecg = 0.6*sin(2*pi*(hr/60)*t) + 0.5*randn(size(t));
```

It simulates normal heartbeats with added random noise.

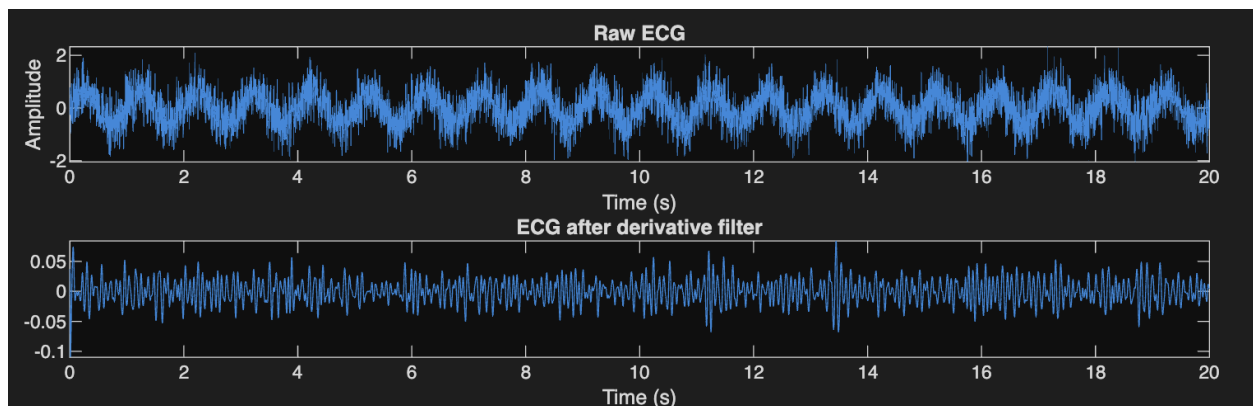
2. Filtering

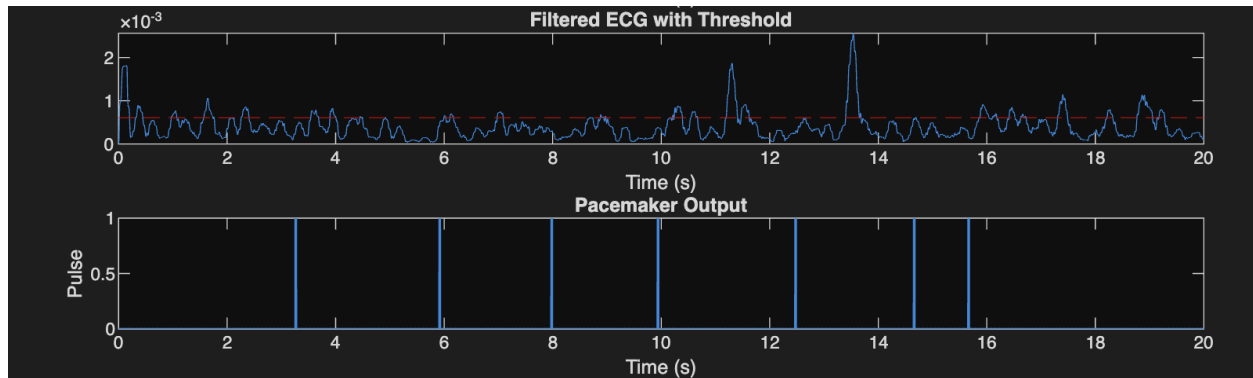
A **2nd-order Butterworth bandpass filter (5–15 Hz)** removes low-frequency drift and high-frequency noise:

```
[b,a] = butter(2,[5 15]/(fs/2),'bandpass');
```

```
filt_ecg = filtfilt(b,a,ecg);
```

`filtfilt` ensures zero-phase distortion.





3. Feature Extraction

To detect heartbeats:

1. **Derivative filter** enhances sharp QRS peaks
`d_ecg = filter([1 2 0 -2 -1]/8,1,filt_ecg);`
2. **Squaring** emphasizes large peaks
`sq_ecg = d_ecg.^2;`
3. **Moving average (150 ms)** smooths the signal
`mov_int = filter(ones(1,win_size)/win_size,1,sq_ecg);`

4. Thresholding

A threshold = $1.5 \times \text{mean signal}$ marks R-peaks:

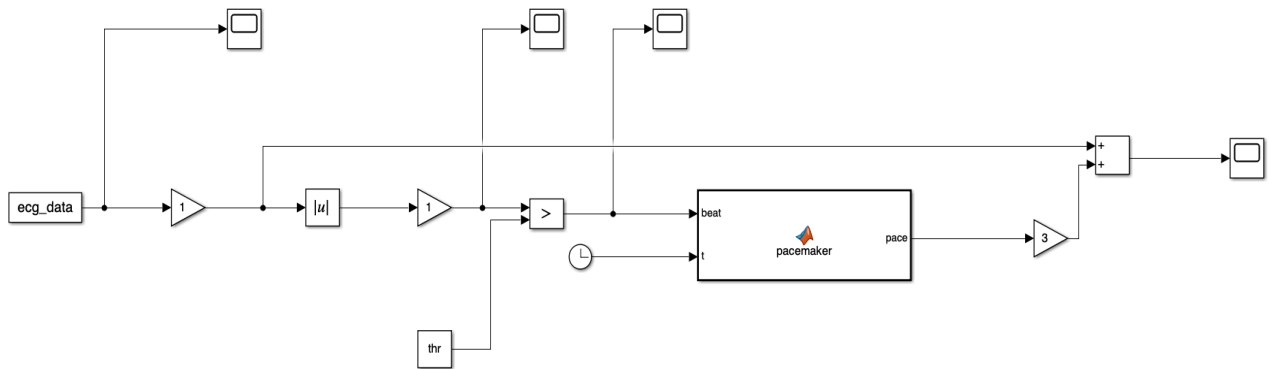
```
r_peaks = mov_int > mean(mov_int)*1.5;
```

5. Pacemaker Logic

If no heartbeat is detected within **1 s (escape time)**, a **2 ms pacing pulse** is generated:

```
if (t - last_beat) > 1.0
    pace = 1; last_beat = t;
end
```

A **refractory period** of 250 ms prevents false detection.



(Here, `ecg_data` is an input from the workspace of the submitted code file; the gain operator can be used to highlight specific parts in the plot.)

6. Output

Four plots are shown:

1. Raw ECG
2. ECG after derivative filter
3. Filtered/thresholded ECG
4. Pacemaker output pulses

The program prints how many times the pacemaker fired.

Conclusion

The code replicates the core logic of a pacemaker:

- Detects heartbeats using digital filtering and thresholding
- Generates pacing pulses when heart rate drops below 60 bpm
- Demonstrates how signal processing and control logic combine to mimic real pacemaker behavior.